An Empirical Study of Machine Learning-based Synthetic Job Trace Generation Methods

Monish Soundar Raj, Thomas MacDougall, Di Zhang, and Dong Dai

{msoundar, tmacdoug, dzhang16, ddai}@charlotte.edu
Computer Science Department,
University of North Carolina at Charlotte, Charlotte NC, USA

Abstract. Job traces play an increasingly important role in today's data-driven and machine learning-based computing system optimizations. However, the lack of sufficient public real-world job traces and the difficulty of publishing more job traces from large computing centers make generating synthetic job traces necessary and important. In this study, we conduct a comparative study of five different machine learning-based data generation methods (i.e., GAN, CTGAN, TVAE, Gaussian Copula, Copula GAN) on three different performance metrics (i.e., CDF plots, statistical metrics, simulation-based scheduling). We also conducted a comparison with the state-of-the-art manual statistical job trace generation method, which was the default method before the new generative models were recently proposed. Our results suggest that many of the machine learning-based data generation methods can consistently produce high-quality job traces without significant manual fine-tuning efforts. The generated trace can be comparable to or even better than the trace generated from highly-optimized manual statistical models. We believe such results indicate the possibility of building an effective pipeline to generate high-fidelity job traces given an arbitrary real-world job trace, allowing for quick and large-scale release of job traces. We provide all the generated traces at: `https://github.com/DIR-LAB/Gen-Parallel-Workloads` for others to use.

Keywords: Job Trace Generation · Generative AI · Workload Analysis

1 Introduction

Large-scale computing clusters are increasingly important today as they run critical applications, such as scientific simulations, financial modeling, and training machine learning models, and consume significant amounts of energy [19, 16]. Thus, their operational efficiency, such as how quickly their jobs are scheduled, day-to-day resource utilization, and overall energy costs, is critical. Here, a job is a general term for computational tasks submitted by users. In heavily loaded clusters, jobs often need to wait in a queue for scheduling and resource provisioning, both of which significantly impact the efficiency of the clusters.

Workload characterizations have proven effective for gaining insights into large-scale computing systems. By analyzing real-world job traces, which are

Table 1: Cluster job trace datasets published in recent years.

Dataset	**Affiliation**	**Years**	**Job Count**	**Nodes Numbers**
Mira [4]	ALCF	2013∼2019	750,000	49,152
Theta [33]	ALCF	2017∼2023	522,858	4,392
Waters [23]	NCSA	2013∼2019	10.5 M	26,864
ThetaGPU [34]	ALCF	2020∼2023	135,975	24
Supercloud [29]	MIT	2021-01∼2021-10	395,914	704
Philly [14]	Microsoft	2017-08∼2017-12	117,325	552
Helios [11]	Sensetime	2020-04∼2020-09	3.3 M	802
Elasticflow [9]	Microsoft	2021-03∼2021-05	69,351	NA
Alibaba Cluster Trace [37]	Alibaba	2023	8,152	1,523

records of job executions in computing clusters, researchers can understand various system characteristics, such as job runtime and wait time distributions, user behaviors, performance bottlenecks, and resource optimization. These insights can further be used to improve system and scheduler designs for better cluster efficiency [27, 24, 6, 36, 1, 17, 28, 35, 11]. As a result, data-driven optimizations based on machine learning methods are increasingly popular [41, 42].

All of these efforts depend on obtaining a large number of job traces for analysis. Recently, various computing cluster sites have made efforts to publish their internal job traces for research purposes. Table 1 summarizes some representative job traces from both academic and industrial clusters. The advantage of using a real-world job trace is that it represents the 'real' execution of the cluster, hence the workload reflects precisely what has been executed.

Although real-world job traces are invaluable for understanding system efficiency, they fall short for several reasons. First, the amount of published job traces is insufficient. Modern data-driven system optimizations require a large volume of job traces to train their machine learning models [41]. While collecting these job traces internally is not necessarily difficult (one can simply dump the SLURM database to obtain the job traces), the anonymization post-processing is often complicated as it could potentially lead to data breaches. For instance, most of the currently published job traces are anonymized using random hashing, which could expose certain users and their workloads if malicious attackers infer the usage pattern of a user. The potential risk of 'unintentional' data leakage has greatly reduced the enthusiasm of data centers to publish their job traces, leading to a lack of sufficient public job traces for machine learning models to learn from or conduct verification on. Second, real-world job traces fall short in simulating hypothetical scenarios ("what-if" situations). These scenarios are crucial for assessing the resilience of existing and new cluster systems under unexpected conditions and aid in developing and testing new scheduling policies.

We believe a model capable of understanding the unique properties of each cluster trace to generate new synthetic data that mirrors the marginal and joint distributions as seen in the original traces would be instrumental. Additionally, such a model provides a means for cluster administrators to produce synthetic job

traces from real job traces for research use without worrying about compromising privacy or sensitive information.

There have been many efforts to generate synthetic job traces using statistical methods [7, 15]. These early works have demonstrated the potential to produce synthetic traces that approximately resemble actual job traces [21]. However, these approaches involve complex pre-processing steps that require significant user involvement (e.g., deciding which distribution to use and determining key parameters of the distribution). The generated traces are hence very sensitive to the quality of decisions made by the users, limiting their usage in the real world. Recently, the rise of Generative AI methods in automatically creating synthetic content across various media, including images [5, 13, 12], videos [10], and tabular data [32], suggests potential relevance in generating job traces. This raises the question: can these new generative models produce high-quality synthetic job traces from learning the internal structures of a given job trace?

This study seeks to conduct a comparative study of different machine learning-based synthetic data generation methods to determine their capability to generate job traces that are similar to a given, original job trace. We also aim to understand if the new machine learning methods can surpass the performance of previous manual statistical models in closely mirroring the distributions of real-world counterparts. To achieve these goals, we employed a comprehensive evaluation pipeline encompassing **five** different machine learning-based methods (GAN, CTGAN, TVAE, Gaussian Copula, Copula GAN) and compared their performance using **three** different metrics: CDF plots, statistical measures, and simulation-based scheduling results, to assess the quality of synthetic job traces. To make the results general, we used **four** modern real-world job traces (i.e., Theta, Blue Waters, Philly, Helios) for the comparisons. These four were selected as they represent both traditional simulation-based HPC workloads and emerging AI/ML workloads. The contributions of this study are twofold:

- To the best of our knowledge, this is the first study to evaluate how generative models perform in generating synthetic job traces.
- We conducted extensive evaluations on how certain models perform across traces under different metrics, providing a solid foundation for the next steps in designing effective methods for generating high-quality job traces.

The remainder of this paper is organized as follows: In §2 we introduce the necessary background about Job Traces and various machine learning-based data generation methods. In §3 we describe the details of data processing and performance metrics calculations. We present the main results in §4. We conclude this paper and discuss the future work in §5.

2 Background

2.1 Job Traces Generation

The *Job Traces* or *Workloads* discussed in this study are essentially a sorted list of job records. Each job record represents a job submitted to the system and

includes various metadata about that job. The key metadata includes: the user (who submitted the job), the number of nodes, CPUs, or GPUs required to run the job, the job submission time, the job run time, the job waiting time, and the final job status (e.g., failed, killed, or successful), etc.

The essential idea of job trace generation is that, given a list of jobs collected from real-world computing clusters, we need to generate another list of jobs with the requirement that the generated jobs represent similar features as the original job trace contains. The simplest approach could be to assume the original job traces follow a certain statistical distribution, such as the gamma distribution, and then find out the best parameters of the distribution by checking moments such as the mean and the variance of the sample job trace. Previous studies have discussed that such a simple method may be misleading because they may not represent the shape of the distribution correctly [21].

2.2 Machine Learning-based Data Generators

Recently, Generative AI methods have attracted significant attentions for generating images [5, 13, 12], videos [10], and even HPC applications' I/O traces [26]. Their ability to generate high-quality datasets suggests the potential to generate job traces. In this study, we investigated five representative generative methods for synthetic job trace generation. Specifically, We studied GAN [8], CTGAN [38], TVAE [38], Copula GAN (CG) [32], and Gaussian Copula (CG) [25] as previous study have shown their effectiveness in generating various datasets [20, 32, 39]. We first provide a brief introduction of each selected method below.

GAN or Generative Adversarial Network [8] is a deep learning architecture serving as the basis for many generative AI methods. Its architecture consists of two separate learning models: a generator and a discriminator. These two models work against each other in a competitive manner that, ideally, ends up in the model generating data that is indistinguishable from real-world data.

More specifically, the generator's goal is to, as the name suggests, generate synthetic data, while the discriminator's role is to separate the generated data from the real ones. Each time, a random noise $p_z(z)$ is used as an input for the generator, its model $G(z;\theta_g)$ is typically composed of several layers of perceptrons, parameterized by θ_g. The discriminator, represented in the same way by $D(x;\theta_g)$ when given some input data x, outputs a single number that represents its confidence that x came from real data, rather than data output by G. D is trained to correctly assign the proper labels to either category of data, while G minimizes the function $(log(1 - D(G(z))$, the chance that D categorizes G output as being fake. Through this min-max game, a GAN model is trained to output convincing synthetic data. However, a major shortcoming of this model is its difficulty in handling complex data. Due to their relatively simple design, GANs struggle to truly understand the relationships between multivariate data, and so have a harder time learning and outputting complex data.

CTGAN, or Conditional Tabular GAN, is a GAN-based model focusing on generating tabular data [39]. It considers that the existing min-max normalization in GANs is insufficient for continuous variable columns in a table. Based

on such an observation, CTGAN uses a Variational Gaussian Mixture (VGM) to normalize the multiple modes of each continuous column separately, if any; A one-hot vector, represented by $\beta_{i,j}$ and a scalar value, represented by $\alpha_{i,j}$ are produced for every column value. The one-hot vector indicates to which mode the specific value belongs, and the scalar value represents the value within the mode. The discrete values are converted to one-hot representation. Then, the concatenation of continuous and discrete column values represents a row like this: $r_j = \alpha_{1,j} \oplus \beta_{1,j} \oplus \ldots \oplus \alpha_{N_c,j} \oplus \beta_{N_c,j} \oplus d_{1,j} \oplus \ldots \oplus d_{N_d,j}$, where $d_{N_d,j}$, is a one-hot representation of a discrete value. CTGAN leverages conditional vectors and training-by-sampling; these two methodologies enhance the representation of less frequent discrete values during training and facilitate the model's ability to understand the relationship between columns.

TVAE or Tabular Variational Auto-Encoder (TVAE) [38], as the name suggests, is a VAE for generating tabular data. First, VAE stands for Variational Auto-Encoder. It is a machine learning model used to generate data by first estimating a training dataset's latent variables; that is, hidden variables found through inference that control how a distribution looks and acts. By estimating these latent variables, VAE can sample from this space and then decode that sample to generate synthetic data. VAE consists of two parts: an encoder, which takes the input data and transforms it into its latent space, and the decoder, which does the exact opposite, taking a sample from that latent space and transforming it back into a form similar to the data fed into the encoder. Typically, the latent space is estimated with a Gaussian distribution, the encoder controlling its shape. Estimating the latent space can be seen as compressing the data to distill it down to its essential, latent variables. Once an estimate of the latent space is obtained, the VAE samples a point along the Gaussian distribution, and the decoder inverses the process the encoder uses in order to transform the compressed sample back into an uncompressed form. Ideally, given some sample input vector x, the decoder outputs a vector $\hat{x}$ that is identical to x. TVAE extends VAE through two modifications: first, by running the input data through the same preprocessing steps as the CTGAN listed above. Additionally, TVAE uses a different loss function. It leverages Evidence Lower-Bound (ELBO) [18] as the loss function. These two changes together make a TVAE better suited for generating tabular data than a typical VAE.

Gaussian Copula is a statistical model used for generating synthetic data [25]. The Gaussian Copula model assumes the data provided has only numerical values. If that is not the case, preprocessing steps will be applied to make everything numerical [25]. Two components construct the generative Gaussian Copula Model: 1) Distribution and 2) Covariance. Numerical columns of the data are analyzed using the Kolmogorov-Smirnov test, which is employed to determine the best-fitting distribution from pre-existing options like the Gaussian Distribution, Truncated Gaussian Distribution, and Uniform Distribution. This test ensures the selected distribution closely matches the observed data. Moreover, the model trainer can select additional distribution options for particular columns, including Beta and Exponential. Consequently, based on the best fit

Table 2: Distribution Parameters In Gaussian Copula Model

Distributions	**Parameters**
Truncated Gaussian Distribution	Mean (μ), Variance (σ^2), Min Value, Max Value
Uniform Distribution	Min Value, Max Value
Beta Distribution	Alpha (α), Beta (β)
Exponential Distribution	Decay Rate (λ)
Gaussian Distribution	Mean (μ) and Variance (σ^2)

from the K-S test and/or distribution set by the trainer, specific parameters are calculated (refer to Table 2). In addition to these distribution parameters, the model also calculates a covariance matrix to understand the joint distribution of the columns.

Copula GAN is a mix of Gaussian Copula and CTGAN. It uses a normalizer from Gaussian Copula to learn the marginal distributions of the real columns; then, the learned marginal distributions are used to normalize the values, creating normal/gaussian curves with $\mu = 0$ and $\sigma = 1$ [31, 30]. Further, the normalized data is given to the CTGAN, and this follows all the steps in the CTGAN section above to train and produce the data.

3 Experiments Design

We have operated on the pipeline architecture illustrated in Figure 1, where the first phase was collecting and pre-processing the raw job traces, phase two was training selected models with the pre-processed data and generating synthetic job traces from the trained models, and the final phase was evaluation. This section provides an in-depth explanation of training data collection, pre-processing, and evaluations utilized in this study.

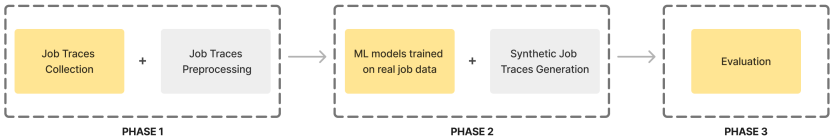


Fig. 1: Workflow pipeline used in this study.

3.1 Job Trace Selection and Preprocessing Phase

To ensure a comprehensive analysis across diverse job handling architectures and to be related to the current job workloads, we systematically collected job traces from four distinct clusters, each publicly available and each job trace no earlier than 2017. Among these are Blue Waters, Theta, Philly, and Helios, as

Table 3: The selected job trace metadata

Cluster Name	Affiliation	Nodes	CPU-Cores	GPUs	Scheduler
Blue Waters	NCSA, UIUC	26864	396000	4228	TORQUE
Theta	ALCF	4392	281088	NA	COBALT
Philly	Microsoft	552	NA	2490	YARN
Helios	Sensetime	802	NA	6416	SLURM

Table 3 shows. Blue Waters [22], formerly operational at the National Center for Supercomputing Applications (NCSA), integrated with 22,636 CPU nodes and 4,228 GPU nodes, represents a hybrid cluster used to run traditional and machine learning workloads. In contrast, Theta [2], a CPU-only cluster hosted in the Argonne Leadership Computing Facility, has 4,392 compute nodes with 64 CPUs each, reflecting a classical high-performance computing (HPC) environment. Recognizing the rise in Artificial Intelligence (AI) and Machine Learning (ML) jobs in recent times, we have collected job traces from Helios [11] and Philly [14], GPU-only clusters - both are dedicated ML job clusters with around 6,416 and 2,490 GPUs, respectively. Helios is operated by SenseTime, and Philly is operated by Microsoft. Helios cluster handles larger jobs than Philly [43].

The raw Job Traces collected from various clusters exhibited heterogeneity in terms of columnar structure and datatype. Each cluster's Job Traces featured unique columns, job counts, and, in some cases, alphanumeric values for user IDs alongside numerical values for Job Status. Additionally, some Job Traces included DateTime values for columns like Job Submit Time. These inconsistencies necessitated preprocessing the data to standardize it before inputting it into machine learning models. We implemented the following preprocessing steps:

- Truncated all job trace datasets to a uniform size of 15000 jobs/rows.
- Converted all the DateTime stamps columns to Unix timestamps.
- Normalized numeric and alphanumeric user IDs by mapping each unique value to a number, ranging from 1 to e, where e represents the number of distinct users in truncated job traces.
- Standardized the representation of Job Status codes across datasets by categorizing them into three distinct groups: Pass, Failed, and Killed. This was necessary because the datasets from Philly and Helios clusters had these categorical divisions exclusively, without listing the numerical status codes.
- Removed columns that were of no importance for the evolution phase, keeping columns - ‘u_id‘, ‘user‘, ‘gpu_num‘, ‘cpu_num‘, ‘node_num‘, ‘interval‘, ‘run_time‘, ‘wall_time‘, ‘new_status‘. Please refer to Table 4 for details on each column.

3.2 Training and Generation Phase

In this study, we have utilized GAN, CTGAN, TVAE, Copula GAN (CG), and Gaussian Copula (GC) for synthetic job trace generation. For the implementation and generation of synthetic data using CTGAN, TVAE, Copula GAN, and

Table 4: Description of Job Trace Columns

No.	Column	Description
1.	u_id	A Integer; A unique identifier assigned to each job for distinguishing purposes.
2.	user	A Integer; User id, an identifier assigned to distinct user.
3.	gpu_num	A Integer; Number of GPUs a job uses.
4.	cpu_num	A Integer; Number of CPUs a job uses.
5.	node_num	A Integer; Number of Nodes a job uses.
6.	interval	In Seconds; Time taken for a job to arrive after the previous job was submitted
7.	run_time	In Seconds; Total time a job was running.
8.	wall_time	In Seconds; Total time a job spent in the system from submit to completion
9.	new_status	A String; Status of the job, when it was completed. Can be either: Pass, Failed, Killed

Gaussian Copula, we employed the Python library `SDV` [30]. Simultaneously, for the GAN model, we utilized the `ydata-synthetic` library [40]. Each model was trained separately on the four job traces, with each job trace comprising 15,000 jobs/rows. Furthermore, we generated 15,000 jobs/rows of synthetic job traces from each trained model to ensure a fair comparison between the real and synthetic job traces.

3.3 Evalution Metrics Phase

In this study, we employ a comprehensive set of metrics to evaluate the quality of synthetic job traces generated by the models in use. These metrics include: (1) Cumulative Distribution Functions (CDFs) and Correlation Heatmaps: this metric provides a comprehensive analysis of both marginal and joint distribution characteristics of the synthetic job traces. CDFs offer a visual and analytical method to compare the distributions of generated data against the real data, emphasizing how closely the synthetic traces replicate the overall distribution trends of each individual field in the original data. Additionally, correlation heatmaps compare the dependencies and relationships between different job trace columns within the training data against those in the synthetic data generated by each model. (2) Several numerical statistical metrics, including the Hellinger Distance, Wasserstein Distance, KL divergence. These metrics quantitatively measure the similarity between the distributions of the generated synthetic job traces and the original job traces. Each metric offers a unique perspective on the comparison, covering aspects like statistical distance, distribution shape, and entropy. (3) Scheduling simulation: this involves running a simulation with the generated job trace to assess scheduling metrics. This practical approach tests how well the synthetic job traces mimic the behavior of real job traces.

Cumulative Distribution Functions (CDF) plots provide an intuitive graphical representation of the distribution/shape of numerical variables (columns), where the x-axis represents the variable's possible values, and the y-axis represents the cumulative probability of the variable being less than or equal to x. In this study, we use CDFs to examine columns from both synthetic and real job traces to see if the synthetic traces converge with the distribution patterns seen in real traces. The columns investigated are: user, interval, cpu_num/gpu_num, and run_time. The CDF plots from the investigation are detailed in the section. 4.1. This approach cultivates an understanding of how well the synthetic traces replicate the marginal distributional properties of real-world traces.

We also calculate several statistical metrics for comparison. First, *Hellinger Distance* is a metric to measure the amount of overlap between two distributions, and, in a sense, gain a measure of how similar two distributions are. For our work, we can convert each trace into a list containing the probability of each distinct value in each column occurring, and then compare them against each other in order to tell if the real and generated traces are statistically similar.

$$H(P,Q) = \frac{1}{\sqrt{2}}\sqrt{\sum_{i=1}^{k}(\sqrt{p_i} - \sqrt{q_i})^2}$$

The formula for the Hellinger Distance takes the summation of the squared difference of the square roots of every pair of points in two probability distributions P and Q, before taking the square root of that summation and dividing it by the square root of two (this normalizes the result). This formula can be thought of as a way to find the total distance between each corresponding point of two distributions. Should the two distributions be identical, the Hellinger distance will be 0, but should they share no overlap, then the distance will be 1.

Second, *Wasserstein Distance* is another measure of distance between two probability distributions, but shares a distinct difference with Hellinger distance in that rather than simply measuring the vertical distance between each pair of points in two distributions, the Wasserstein distance instead measures how much work it would take to transform one distribution into another:

$$W_p(P,Q) = \int |x-y| d\gamma(x,y)$$

$$\gamma \in \Gamma(P,Q)$$

The formula for the Wasserstein distance takes a joint distribution γ of two probability distributions P and Q. Then, for two positions along that joint distribution x and y, we take the absolute value of their difference. We take the integral of this process repeated across every possible point in γ. Intuitively, this process can be thought of as the 'cost' of filling in a hole shaped like P with a pile of dirt shaped like Q, where a low cost indicates similar distributions.

Third, *Kullback–Leibler (KL) Divergence* is another similarity metric. Essentially, it calculates the difference in information contained between two different

distributions. Given a distribution existing within a sample space X, the formula for entropy is as follows:

$$H(X) = -\sum_{x \in X} P(x) \log P(x)$$

Essentially, for each point x in our distribution, this formula takes the probability of x and multiplies it by the log of itself, summing all these together to obtain the entropy of our distribution. To now find the KL divergence of two different distributions P and Q, we only have to modify the formula slightly:

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log(\frac{P(x)}{Q(x)})$$

This modification informs us now of the difference in information contained within P and Q. The KL divergence gives us another heuristic to measure how similar two distributions are, and can be interpreted as how much 'surprise', or uncertainty, we might have if using P as an estimate of Q. So, lower is better.

Last, we compared original and generated job traces by directly running them through a scheduler simulator (i.e., SchedGym [41]). In the simulator, we used the basic and popular First-Come-First-Serve (FCFS) scheduler policy with EASY Backfilling to run both the original and generated job traces and calculate one key scheduling result: *System Utilization*, which is also called utilization rate, represents the average percentage of compute nodes allocated normalized by the entirety of nodes in the system over a given period of time.

4 Results and Discussions

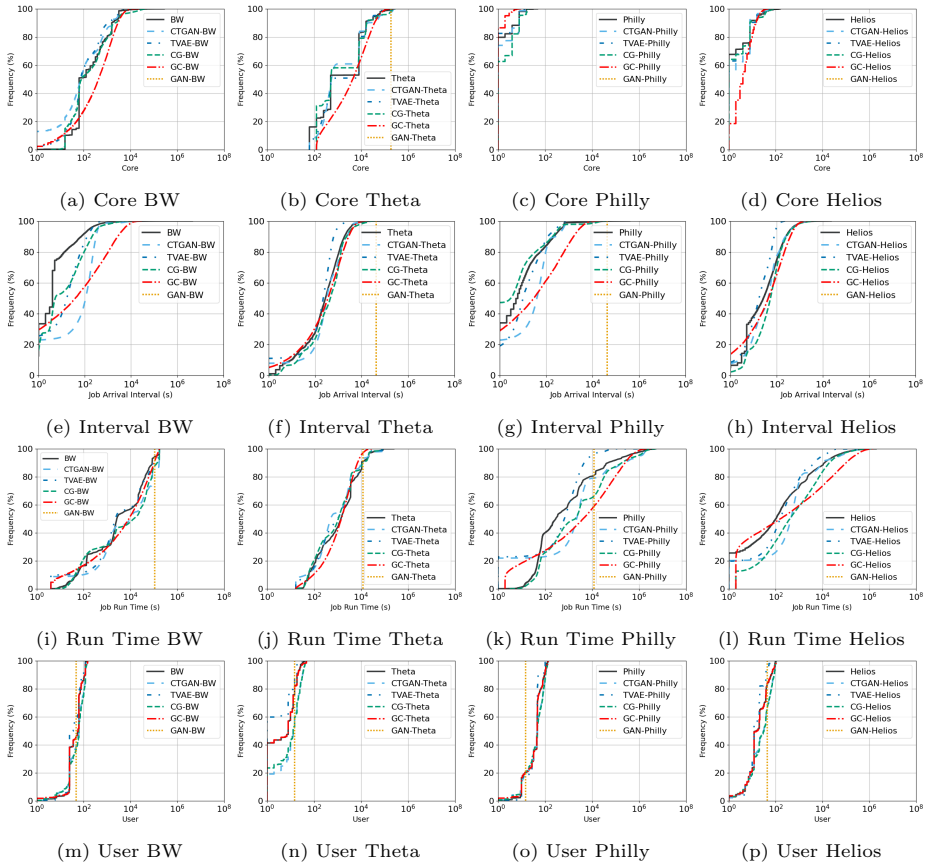
4.1 Machine Learning Model

Our first set of experimental results is to show how different machine learning-based methods perform in generating synthetic job traces.

Using five different machine learning-based data generators and four selected original job traces, we generated 20 synthetic job traces. Each job in the synthetic job traces consists of multiple metadata fields, such as user (who submitted the job), core usage (how many cores it uses), submission time (when the job gets submitted), and runtime (how long the job will run), etc.

We compare these fields individually. Note that, we translate the submission time to job submission interval for the comparison as the interval is typically more interesting and useful for designing job scheduling policies.

Figure 2 shows the overall results. Here, each row indicates one particular metadata field of the job, for instance, the first row shows the Core usage. In each row, we show the results of the four selected job traces (BW indicates BlueWaters). In each plot, we plot both the CDF of the original job trace (solid line) and the CDFs of all the generated job traces (dotted lines). Here, CG indicates Copula GAN and GC indicates Gaussian Copula. Note that, for the



(a) Core BW (b) Core Theta (c) Core Philly (d) Core Helios

(e) Interval BW (f) Interval Theta (g) Interval Philly (h) Interval Helios

(i) Run Time BW (j) Run Time Theta (k) Run Time Philly (l) Run Time Helios

(m) User BW (n) User Theta (o) User Philly (p) User Helios

Fig. 2: CDFs of *Core Usage*, *Interval*, *Job Run Time*, and *User* of different machine learning models with real and synthetic job traces.

Core usage row, BW results are the sum of both CPU and GPU usage; Theta only contains CPU usage; Helios and Philly only contain GPU usage.

Across all the plots, we can observe that the CDF plots of Copula GAN, TVAE, and CTGAN models are generally closer to that of the original job trace, outperforming other models, i.e., the Gaussian Copula and vanilla GAN. In fact, we can observe that in many cases, the vanilla GAN performs extremely bad. Their CDFs are simply vertical lines, indicating the model is not able to generate anything meaningful. By checking the actual generated job traces, we noticed that the model produced identical job data across the entire job trace. We believe this is likely caused by the "mode collapse" [3] during the training, a phenomenon where the discriminator performs very well, and the generator performs worse by producing limited non-diverse data.

It is also interesting to observe that the machine learning models perform quite differently across different job metadata fields. For instance, for Core usage,

Table 5: Statistic Metrics across job traces and generation methods.

Job Trace	Metric	CG	CTGAN	GC	TVAE	GAN
Blue Waters	Helli	0.381	0.362	0.420	**0.314**	0.971
	Wass	0.0201	0.020	0.00788	**0.00459**	0.00928
	KL	11.1	4.0	11.7	**3.06**	62.2
Theta	Helli	0.460	0.453	0.435	**0.408**	0.942
	Wass	0.0197	0.0238	**0.00107**	0.0140	0.0223
	KL	12.8	**8.99**	14.2	9.75	58.9
Philly	Helli	0.326	0.246	0.333	**0.230**	0.847
	Wass	0.0206	0.0141	0.0362	**0.00429**	0.0830
	KL	12.5	**2.50**	12.7	3.24	57.9
Helios	Helli	0.289	0.219	0.305	**0.205**	0.662
	Wass	0.0384	0.0325	**0.00750**	0.00852	0.0573
	KL	6.95	**1.04**	7.42	2.58	40.9

Run Time, and User, many of the machine learning models perform quite well to fit the original CDFs. But for Interval, many of the models won't be able to generate high-quality traces, especially for plots (e) and (g). We believe this is caused by the abnormality of certain job metadata fields. For instance, Blue Waters has some big gaps in its Job Submission Time. There are periods of weeks (e.g., 51 days) without any new job submitted. This creates a small number of huge outliers in the Interval field, effectively impacting the learning accuracy. We will discuss more on this in a later subsection.

By checking all the plots across different job metadata fields and different workloads, we can see that **TVAE, CTGAN and Copula GAN** perform consistently better than other options and inherently the best ML-based job trace generation methods from fitting the CDF point of view.

We further check the statistical metrics described in Section 3.3. Table 5 shows the results. Here, we show the metrics of *Hellinger Distance*, *Wasserstein Distance*, and *KL Divergence* (Represented as *Helli*, *Wass*, and *KL*, respectively.) between the original job traces and the generated job traces using different methods. From these results, we can see that among all models, most metrics fall within a similar range, indicating a good similarity except vanilla GAN, which performs notably worse and gets the worst scores in all three categories, including an egregiously high average KL divergence. This falls in line with our CDF results about the GAN model. GAN's worst performance in both the CDFs and statistical metrics can be attributed to the collapse of the model when trying to train with HPC job traces, which led to a scenario where data produced by the GAN synthetic data only had a single row replicated across the entire job trace. In contrast, the **TVAE** received the best average scores in KL divergence and Hellinger distance.

In the last, the simulator-scheduler scores, as detailed in Table 6, show, on average, that the TVAE model performs the best across different workloads, with CTGAN scoring the second best. The Copula GAN (CG) performs extremely

Table 6: Util score after scheduling original job trace and generated traces

Job Trace	**CG**	**CTGAN**	**GC**	**TVAE**	**Origin**
Blue Waters	0.89	**0.86**	0.91	0.91	0.82
Theta	0.25	**0.44**	0.24	0.95	0.61
Philly	**0.81**	0.32	0.60	0.95	0.84
Helios	0.70	0.66	0.80	**0.82**	0.97

Table 7: Outlier Summary for Interval Column Across Datasets

Dataset	**'*Outlier*' Rows**	**'*Outlier*' Percentage**
Blue Waters	3,004	20.03%
Theta	1,343	8.95%
Philly	2,747	18.31%
Helios	1,516	10.11%

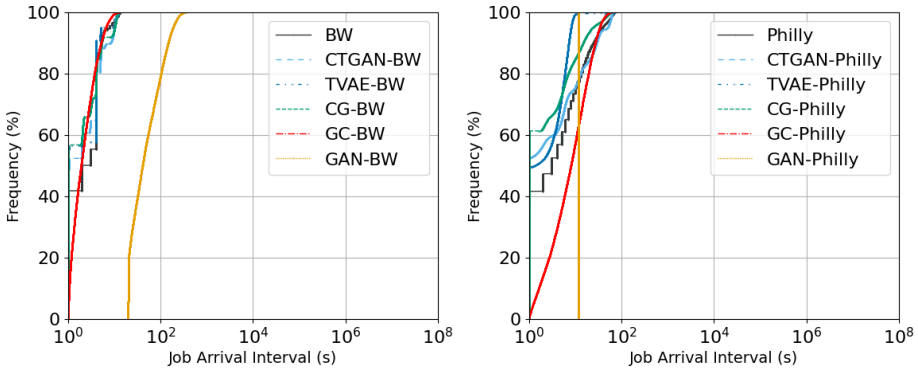
well in Blue Water, Philly, and Helios but encounters some issues in Theta, failed to maintain its best machine learning model ranking. This clearly shows the challenge of building a highly generic job trace generation pipeline. On the other hand, we have to note that the scheduling results are much more sensitive to the generated job traces. it is very possible that a small change in job trace leads to high variance in the scheduling sequence and hence the final utilization score. The results, however, do suggest the high quality of generated job traces from these machine learning models. Unfortunately, due to the GAN model producing identical job data across the entire job trace, we could not score it.

4.2 Improving Interval CDFs

To understand the performance discrepancy in the Interval CDF plots reported in Figure 2, such as Blue Waters and Philly Interval, we calculate the '*Outliers*', using The Interquartile Range (IQR), for the particular metadata field. The results are reported in Table 7. From these results, we can observe that the percentage of outliers in the interval column was high in BW and Philly. After removing these interval outliers in BW and Philly, we then observed the subsequent training and plotting showed better interval CDF lines for CTGAN, TVAE, and Copula GAN models (refer to Figure 3). Such results suggest the complexity of generating synthetic job traces as outliers may be dominating the traces but are also hard to learn in an accurate way.

4.3 Machine Learning Model Correlation Heatmaps

In the previous set of evaluations, we have shown the similarity of the generated traces on each individual job field, such as runtime, submission time, and core usage. Given that different job fields often exhibit strong correlations, we further use correlation heatmaps to quantitatively visualize the relationships



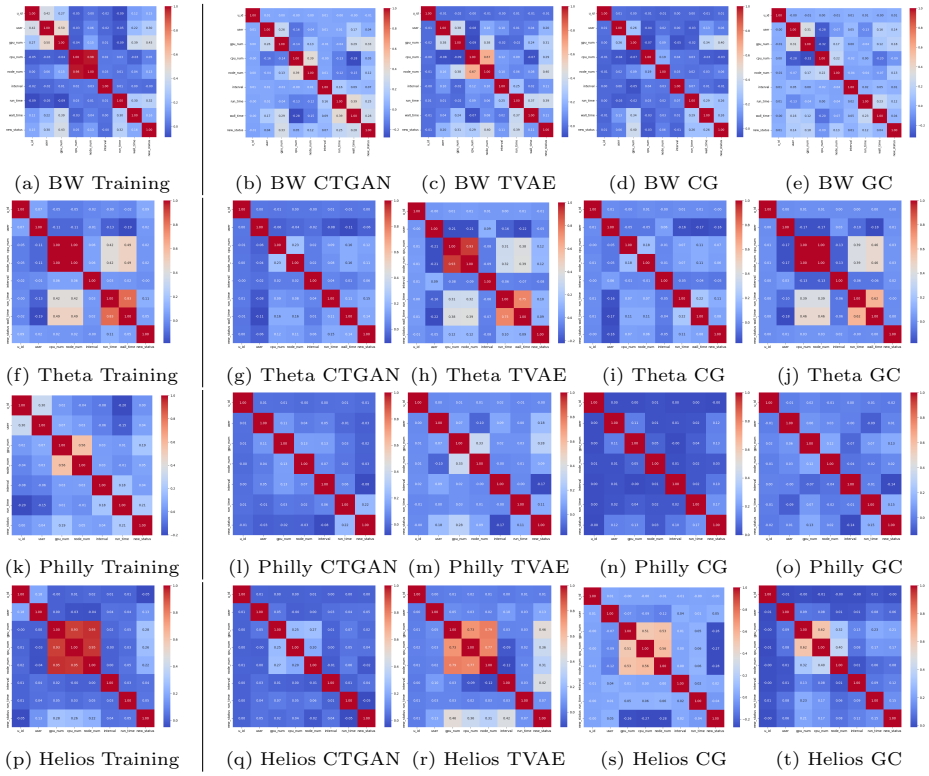
(a) Interval BW without outliers (b) Interval Philly without outliers

Fig. 3: The CDF plots of Interval after removing the outliers in BW and Philly.

between various job trace columns, including u_id, user, gpu_num, cpu_num, node_num, interval, run_time, wall_time, and new_status. These heatmaps display correlation coefficients ranging from -1 to 1, where 1 indicates a perfect positive correlation, -1 indicates a perfect negative correlation, and 0 signifies no correlation. For example, a strong correlation of 0.98 between node_num and cpu_num (shown in Figure 4(a)) suggests that jobs requiring more CPUs also tend to use more nodes.

Figure 4 contains a series of heatmaps for different datasets including, Blue Waters, Theta, Philly, and Helios. Each row starts with the heatmap generated from the original training data, serving as a baseline for assessing the fidelity of synthetic traces. Subsequent heatmaps in each row, created by models like CTGAN, TVAE, Copula GAN (CG), and Gaussian Copula (GC), aim to mirror this original heatmap closely. Such alignment underscores the models' ability to accurately replicate the complex dependencies inherent in the original job traces. Note that, in these plots, we exclude columns that are not included in certain job traces. For instance, the cpu_num column is removed from the Philly datasets because the original data trace does not have CPU counts. The wall_time column is omitted from both the Philly and Helios datasets for the same reason. Similarly, we exclude the gpu_num column from Theta datasets.

From these results, we have several observations. First, none of the heatmaps derived from the synthesized data align perfectly with the original data correlation heatmaps, showing the complexity of capturing the correlations. Second, among all the models, the heatmaps derived from the TVAE model more closely resemble those from the original data, showcasing its superior ability to capture and reproduce high-fidelity column relationships. CTGAN demonstrates second best performance. Note that, the heatmaps for the vanilla GAN model are not included in this series. This is attributed to the mode collapse problem, as discussed previously, which resulted in the GAN generating non-diverse, uniform outputs.



(a) BW Training (b) BW CTGAN (c) BW TVAE (d) BW CG (e) BW GC

(f) Theta Training (g) Theta CTGAN (h) Theta TVAE (i) Theta CG (j) Theta GC

(k) Philly Training (l) Philly CTGAN (m) Philly TVAE (n) Philly CG (o) Philly GC

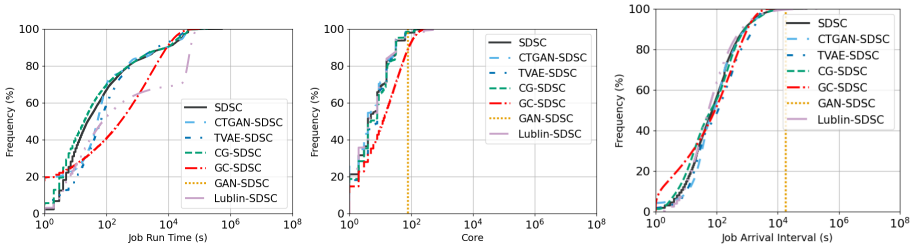
(p) Helios Training (q) Helios CTGAN (r) Helios TVAE (s) Helios CG (t) Helios GC

Fig. 4: Correlation heatmaps comparing the joint distributions of training data and synthetic data generated by different models (CTGAN, TVAE, CG, GC) across four datasets: BW, Theta, Philly, and Helios.

4.4 Comparison with Manual Statistical Model

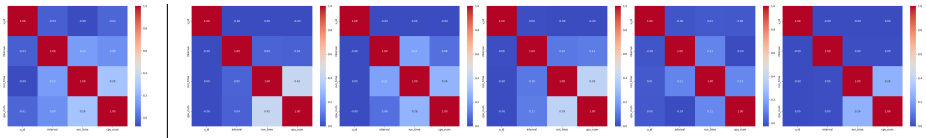
There have been a number of studies using statistical models to generate synthetic job traces. The state-of-the-art solution is Lublin [21]. To understand how the machine learning-based models perform against the statistical models, we further compared them with the Lublin model based on a dataset (San Diego Supercomputer Center Intel Paragon machine, SDSC-95) used in the original Lublin paper. The reason for using such an old dataset is relevant with the main limitation of manual statistical model: their parameters are extremely hard to configure to generate high quality job traces. We used the original job trace, so that we can directly use the same configurations reported in the original paper for comparison.

Our machine learning-based models were trained in the same way on SDSC-95 as previously discussed. Each model then generates **76872** jobs, matching what the original job trace has. We then plotted CDFs of job run time, job core usage, job arrival intervals, as shown in Figure 5, and presented correlation heatmaps in Figure 6, mirroring the analyses previously conducted. The user



(a) Run Time SDSC-95 (b) Core SDSC-95 (c) Interval SDSC-95

Fig. 5: CDF comparisons of job characteristics on SDSC-95 between machine learning-based models and Lublin model.



(a) Training (b) CTGAN (c) TVAE (d) CG (e) GC (f) Lublin

Fig. 6: Correlation heatmaps comparing the joint distributions in the SDSC-95 dataset between the original data and synthetic data. Panel (a) displays the heatmap for the original dataset; Panels (b) through (f) show heatmaps generated by the machine learning models CTGAN, TVAE, CG, GC, and the Lublin model, respectively.

CDF was omitted due to the absence of user data in the SDSC-95 job trace. The results demonstrate that three machine learning models, i.e., TVAE, Copula GAN (CG), and CTGAN, perform either better or comparably to the Lublin model in terms of CDFs. The correlation heatmaps reveal that the TVAE and CG models more accurately reflect the underlying inter-column relationships observed in the original job traces compared to the Lublin model. These models demonstrate superior capability in capturing complex data relationships, which is accomplished without the intensive manual parameter tuning required by traditional methods. This suggests that their synthesized job traces not only surpass those generated by conventional statistical approaches in terms of fidelity but also in their relevance to the actual data.

5 Conclusion and Future Plan

Generating synthetic job traces is increasingly important in supporting data-driven system optimization. In this study, we conduct a comparative study over the performance of various machine learning-based data generation methods. Our study highlights the potential of several models, specifically TVAE, Copula GAN, and CTGAN, in synthesizing high-quality job traces that closely mirror real-world job traces. Among them, **TVAE and CTGAN** consistently perform well across job traces and metrics. We also show these models not only learn

the job trace generation automatically, but also generate job traces with higher accuracy than state-of-the-art statistical approaches, such as the Lublin model. However, our results also reveal some limitations in the machine learning-based models' ability to 1) handle outliers and 2) capture the complex joint relationships among multiple job fields.

Looking forward, we plan to first solve the problem posed by job traces with a high rate of outliers in different columns. These outliers, innate to real-world job traces, present a noteworthy load for ML models, potentially hindering their learning process and influencing the quality of the synthetic job traces. Simply removing outliers, as done in this study for investigation purposes, is not a feasible solution. We also plan to improve the ability to produce synthetic data that performs better in joint distribution by understanding the complex relationships present in job traces. Such advancements could offer more nuances in synthetic job traces.

References

1. Amvrosiadis, G., Park, J.W., Ganger, G.R., Gibson, G.A., Baseman, E., DeBardeleben, N.: On the diversity of cluster workloads and its impact on research results. In: 2018 {USENIX} Annual Technical Conference ({USENIX}{ATC} 18). pp. 533–546 (2018)
2. Argonne Leadership Computing Facility: Anl-alcf-djc-theta dataset [20230101_20231231]. `https://reports.alcf.anl.gov/data/ANL-ALCF-DJC-THETA\_20230101\_20231231.html` (2023), accessed: 2023
3. Bau, D., Zhu, J.Y., Wulff, J., Peebles, W., Strobelt, H., Zhou, B., Torralba, A.: Seeing what a gan cannot generate. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 4502–4511 (2019)
4. Collins, J.: Passing the torch from intrepid to mira. https://www.alcf.anl.gov/news/passing-torch-intrepid-mira (2023), `https://www.alcf.anl.gov/news/passing-torch-intrepid-mira`
5. Dhariwal, P., Nichol, A.: Diffusion models beat gans on image synthesis. Advances in neural information processing systems **34**, 8780–8794 (2021)
6. Di, S., Guo, H., Pershey, E., Snir, M., Cappello, F.: Characterizing and understanding hpc job failures over the 2k-day life of ibm bluegene/q system. In: 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 473–484. IEEE (2019)
7. Feitelson, D.G.: Packing schemes for gang scheduling. In: workshop on job scheduling strategies for parallel processing. pp. 89–110. Springer (1996)
8. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N., Weinberger, K. (eds.) Advances in Neural Information Processing Systems. vol. 27. Curran Associates, Inc. (2014)
9. Gu, D., Zhao, Y., Zhong, Y., Xiong, Y., Han, Z., Cheng, P., Yang, F., Huang, G., Jin, X., Liu, X.: Elasticflow: An elastic serverless training platform for distributed deep learning. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. pp. 266–280 (2023)

10. Harvey, W., Naderiparizi, S., Masrani, V., Weilbach, C., Wood, F.: Flexible diffusion modeling of long videos. Advances in Neural Information Processing Systems **35**, 27953–27965 (2022)
11. Hu, Q., Sun, P., Yan, S., Wen, Y., Zhang, T.: Characterization and prediction of deep learning workloads in large-scale gpu datacenters. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1–15 (2021)
12. Huang, H., Yu, P., Wang, C.: An introduction to image synthesis with generative adversarial nets. arxiv 2018. arXiv preprint arXiv:1803.04469 (2018)
13. Islam, A., Belhaouari, S.B.: Fast and efficient image generation using variational autoencoders and k-nearest neighbor oversampling approach. IEEE Access **11**, 28416–28426 (2023)
14. Jeon, M., Venkataraman, S., Phanishayee, A., Qian, J., Xiao, W., Yang, F.: Analysis of {Large-Scale}{Multi-Tenant}{GPU} clusters for {DNN} training workloads. In: 2019 USENIX Annual Technical Conference (USENIX ATC 19). pp. 947–960 (2019)
15. Jette, M.: Improved utilization and responsiveness with gang scheduling. Tech. rep., Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States) (1996)
16. Jia, W., Wang, H., Chen, M., Lu, D., Lin, L., Car, R., Weinan, E., Zhang, L.: Pushing the limit of molecular dynamics with ab initio accuracy to 100 million atoms with machine learning. In: SC20: International conference for high performance computing, networking, storage and analysis. pp. 1–14. IEEE (2020)
17. Joubert, W., Su, S.Q.: An analysis of computational workloads for the ornl jaguar system. In: Proceedings of the 26th ACM international conference on Supercomputing. pp. 247–256 (2012)
18. Kingma, D.P., Welling, M.: Auto-encoding variational bayes (2022)
19. Kurth, T., Treichler, S., Romero, J., Mudigonda, M., Luehr, N., Phillips, E., Mahesh, A., Matheson, M., Deslippe, J., Fatica, M., et al.: Exascale deep learning for climate analytics. In: SC18: International conference for high performance computing, networking, storage and analysis. pp. 649–660. IEEE (2018)
20. Lee, J., Lee, O.: Ctgan vs tgan? which one is more suitable for generating synthetic eeg data. Journal of Theoretical and Applied Information Technology **99**(10) (2021)
21. Lublin, U., Feitelson, D.G.: The workload on parallel supercomputers: modeling the characteristics of rigid jobs. Journal of Parallel and Distributed Computing **63**(11), 1105–1122 (2003)
22. National Center for Supercomputing Applications: Data sets - blue waters (2023), `https://bluewaters.ncsa.illinois.edu/data-sets`, accessed: 2023
23. NCSA: Blue waters data sets. https://bluewaters.ncsa.illinois.edu/data-sets (2023), `https://bluewaters.ncsa.illinois.edu/data-sets`
24. Patel, T., Liu, Z., Kettimuthu, R., Rich, P., Allcock, W., Tiwari, D.: Job characteristics on large-scale systems: long-term analysis, quantification, and implications. In: SC20: International conference for high performance computing, networking, storage and analysis. pp. 1–17. IEEE (2020)
25. Patki, N., Wedge, R., Veeramachaneni, K.: The synthetic data vault. In: 2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA). pp. 399–410. IEEE (2016)
26. Paul, A.K., Choi, J.Y., Karimi, A.M., Wang, F.: Machine learning assisted hpc workload trace generation for leadership scale storage systems. In: Proceedings of

the 31st International Symposium on High-Performance Parallel and Distributed Computing. pp. 199–212 (2022)

27. Rodrigo, G.P., Östberg, P.O., Elmroth, E., Antypas, K., Gerber, R., Ramakrishnan, L.: Towards understanding hpc users and systems: a nersc case study. Journal of Parallel and Distributed Computing **111**, 206–221 (2018)
28. Rodrigo Álvarez, G.P., Östberg, P.O., Elmroth, E., Antypas, K., Gerber, R., Ramakrishnan, L.: Hpc system lifetime story: Workload characterization and evolutionary analyses on nersc systems. In: Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing. pp. 57–60 (2015)
29. Samsi, S., Weiss, M.L., Bestor, D., Li, B., Jones, M., Reuther, A., Edelman, D., Arcand, W., Byun, C., Holodnack, J., et al.: The mit supercloud dataset. In: 2021 IEEE High Performance Extreme Computing Conference (HPEC). pp. 1–8. IEEE (2021)
30. sdv-dev: SDV: Synthetic Data Vault. `https://github.com/sdv-dev/SDV` (2023)
31. SDV Developers: Copulagan synthesizer – sdv docs. `https://docs.sdv.dev/sdv/single-table-data/modeling/synthesizers/copulagansynthesizer` (2023), accessed: 2023-02-25
32. Synthetic Data Vault (SDV) Development Team: Copulagan synthesizer - sdv documentation. `https://docs.sdv.dev/sdv/single-table-data/modeling/synthesizers/copulagansynthesizer` (2023)
33. Theta: Theta. https://reports.alcf.anl.gov/data/theta.html (2023), `https://reports.alcf.anl.gov/data/theta.html`
34. ThetaGPU: Thetagpu. https://reports.alcf.anl.gov/data/thetagpu.html (2023), `https://reports.alcf.anl.gov/data/thetagpu.html`
35. Versluis, L., Mathá, R., Talluri, S., Hegeman, T., Prodan, R., Deelman, E., Iosup, A.: The workflow trace archive: Open-access data from public and private computing infrastructures. IEEE Transactions on Parallel and Distributed Systems **31**(9), 2170–2184 (2020)
36. Wang, F., Oral, S., Sen, S., Imam, N.: Learning from five-year resource-utilization data of titan system. In: 2019 IEEE International Conference on Cluster Computing (CLUSTER). pp. 1–6. IEEE (2019)
37. Weng, Q., Yang, L., Yu, Y., Wang, W., Tang, X., Yang, G., Zhang, L.: Beware of fragmentation: Scheduling {GPU-Sharing} workloads with fragmentation gradient descent. In: 2023 USENIX Annual Technical Conference (USENIX ATC 23). pp. 995–1008 (2023)
38. Xu, L., Skoularidou, M., Cuesta-Infante, A., Veeramachaneni, K.: Modeling tabular data using conditional gan. In: Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., Garnett, R. (eds.) Advances in Neural Information Processing Systems. vol. 32. Curran Associates, Inc. (2019)
39. Xu, L., Skoularidou, M., Cuesta-Infante, A., Veeramachaneni, K.: Modeling tabular data using conditional gan. Advances in neural information processing systems **32** (2019)
40. ydataai: YData Synthetic: Synthetic Data Generation. `https://github.com/ydataai/ydata-synthetic` (2023)
41. Zhang, D., Dai, D., He, Y., Bao, F.S., Xie, B.: Rlscheduler: an automated hpc batch job scheduler using reinforcement learning. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1–15. IEEE (2020)

42. Zhang, D., Dai, D., Xie, B.: Schedinspector: A batch job scheduling inspector using reinforcement learning. In: Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing. pp. 97–109. HPDC '22, Association for Computing Machinery, New York, NY, USA (2022)
43. Zhang, D., Soundar Raj, M., Xie, B., Di, S., Dai, D.: Cross-system analysis of job characterization and scheduling in large-scale computing clusters. In: 2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE (2024)